
TEMAS DE DISEÑO WEB

Carlos Fernando Vásquez Guerra



WCAG: WEB CONTENT ACCESSIBILITY GUIDELINES

WCAG (Pautas de Accesibilidad para el Contenido Web) son un conjunto de pautas esenciales para garantizar que el contenido web sea **accesible para todas las personas**, incluidas aquellas con discapacidades visuales, auditivas, cognitivas

- Publicadas por el **W3C**, la organización que define estándares web.
- Su objetivo: que los sitios web y aplicaciones sean **usables por el mayor número de personas posible**.

WCAG: WEB CONTENT ACCESSIBILITY GUIDELINES

Las WCAG se estructuran bajo 4 principios fundamentales

P
erceivable

Perceptible: La información debe ser **visible y audible** para todos.
Ejemplo: texto alternativo en imágenes, subtítulos en videos, contraste suficiente de colores.

O
perable

Operable: Los usuarios deben poder **navegar e interactuar** con la interfaz.
Ejemplo: navegación por teclado, menús accesibles, evitar contenido que parpadee demasiado.

U
nderstandable

Comprensible: La información y la interfaz deben ser **claras y predecibles**.
Ejemplo: instrucciones claras, consistencia en botones y etiquetas.

R
obust

Robusto: El contenido debe ser **compatible con distintos navegadores, dispositivos y tecnologías asistivas**.
Ejemplo: sitios que funcionan con lectores de pantalla, navegadores antiguos y dispositivos móviles.

WCAG: WEB CONTENT ACCESSIBILITY GUIDELINES

Se manejan diferentes niveles de conformidad:



Minimum Level: **A**

A (básico) → requisitos mínimos de accesibilidad.

Medium Level: **AA**

(includes A & AA)

AA (medio, recomendado) → la mayoría de los sitios deberían cumplirlo.

Maximum Level: **AAA**

(includes A, AA, & AAA)

AAA (alto) → máximo nivel de accesibilidad, más difícil de lograr.

WCAG: WEB CONTENT ACCESSIBILITY GUIDELINES

Ejemplos

P

Texto alternativo para imágenes → ``

Subtítulos y transcripciones para audio y video.

Contraste suficiente de colores → texto oscuro sobre fondo claro o viceversa.

No depender solo del color para transmitir información → usar íconos, subrayados o patrones.

O

Navegación por teclado → todos los botones y enlaces accesibles con TAB y ENTER.

Tiempo suficiente para leer o completar tareas.

Evitar contenido parpadeante que pueda provocar epilepsia o mareos.

Menús y formularios accesibles → uso claro de etiquetas y campos de formulario.

U

Lenguaje simple y consistente en botones y menús.

Instrucciones claras → por ejemplo, para completar formularios o pasos de un proceso.

Consistencia visual → el mismo tipo de botones hace lo mismo en toda la web.

Retroalimentación de errores clara → mensajes explicativos en formularios.

R

Compatible con **lectores de pantalla y tecnologías asistivas**.

Uso de **HTML semántico** (`<header>`, `<nav>`, `<main>`, `<footer>`).

Código limpio que funcione en **distintos navegadores y versiones**.

Evitar dependencias exclusivas de scripts que no sean estándar.

WCAG: WEB CONTENT ACCESSIBILITY GUIDELINES

Ejemplos

P

Texto alternativo para imágenes → ``

Subtítulos y transcripciones para audio y video.

Contraste suficiente de colores → texto oscuro sobre fondo claro o viceversa.

No depender solo del color para transmitir información → usar íconos, subrayados o patrones.

O

Navegación por teclado → todos los botones y enlaces accesibles con TAB y ENTER.

Tiempo suficiente para leer o completar tareas.

Evitar contenido parpadeante que pueda provocar epilepsia o mareos.

Menús y formularios accesibles → uso claro de etiquetas y campos de formulario.

U

Lenguaje simple y consistente en botones y menús.

Instrucciones claras → por ejemplo, para completar formularios o pasos de un proceso.

Consistencia visual → el mismo tipo de botones hace lo mismo en toda la web.

Retroalimentación de errores clara → mensajes explicativos en formularios.

R

Compatible con **lectores de pantalla y tecnologías asistivas**.

Uso de **HTML semántico** (`<header>`, `<nav>`, `<main>`, `<footer>`).

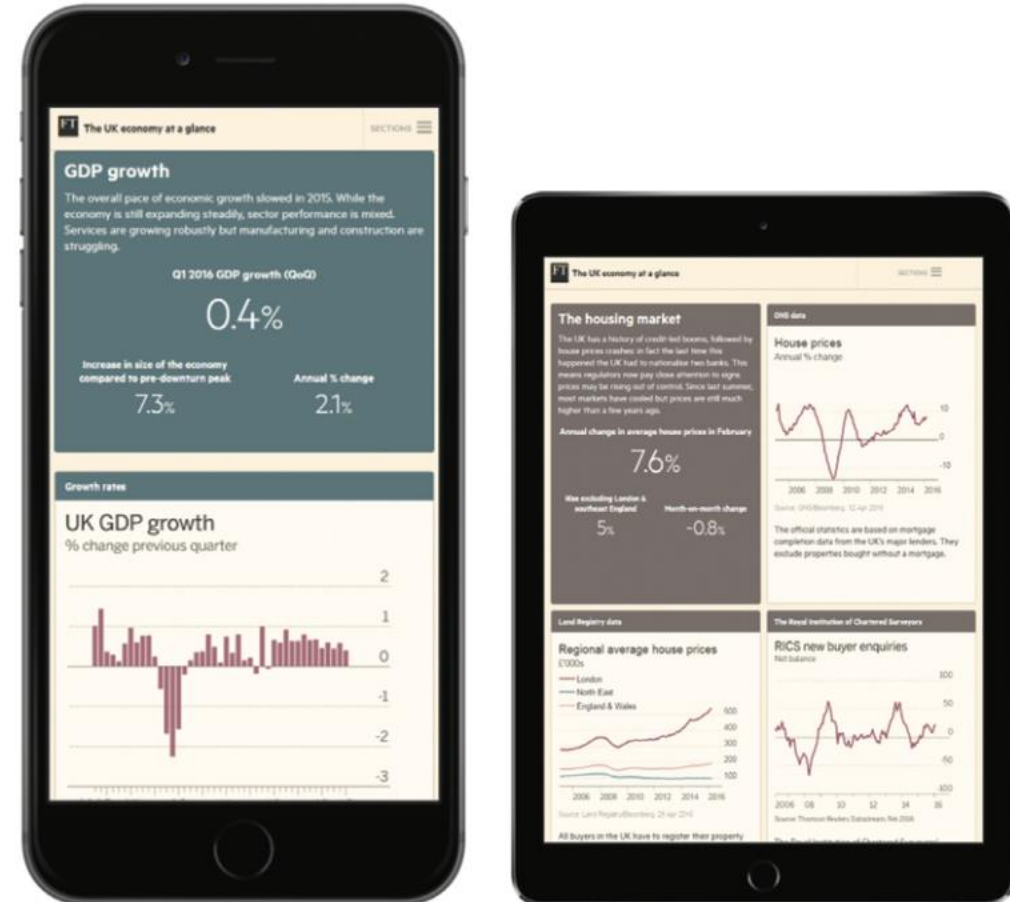
Código limpio que funcione en **distintos navegadores y versiones**.

Evitar dependencias exclusivas de scripts que no sean estándar.

DISEÑO RESPONSIVO

Un **diseño responsivo** (responsive design) significa que una página web o visualización se adapta automáticamente al tamaño y características del dispositivo en el que se ve (computadora de escritorio, tableta, móvil, incluso pantallas grandes).

La idea es que el contenido siga siendo legible, usable y estético, sin necesidad de hacer zoom o desplazamientos incómodos.



DISEÑO RESPONSIVO

- **Diseño flexible con rejillas (grid y flexbox)**
 - Usar **unidades relativas** (% , em, rem, vw, vh) en lugar de píxeles fijos.
 - Estructurar los elementos con **CSS Grid o Flexbox** para que se adapten.
- **Medios adaptativos (media queries)**
 - Usar @media en CSS para definir estilos específicos según anchos de pantalla
- **Imágenes y gráficos escalables**
 - Usar max-width: 100% en imágenes para que no se desborden.
 - Considerar gráficos en **SVG** para que no pierdan calidad.
- **Tipografía adaptable**
 - Usar tamaños de letra relativos (**em, rem, clamp()**) que cambien con la pantalla.
- **Jerarquía clara y simplificación en móviles**
 - En pantallas pequeñas, ocultar elementos secundarios o reorganizarlos.
 - Menús desplegados para navegación.
- **Pruebas en múltiples dispositivos y navegadores**
 - Simular móviles en el navegador (DevTools) o probar físicamente en distintos dispositivos.

DISEÑO RESPONSIVO

Diseño flexible con rejillas (grid y flexbox)

- Usar **unidades relativas** (% , em, rem, vw, vh) en lugar de píxeles fijos.
- Estructurar los elementos con **CSS Grid o Flexbox** para que se adapten.

vw: viewport width

porcentaje del ancho de la ventana

vh: viewport height

porcentaje de la altura de la ventana.

%

Relativa al elemento contenedor padre (no necesariamente al viewport)

```
section {  
  height: 50vh; /* 50% de la altura de la ventana */  
  width: 80vw; /* 80% del ancho de la ventana */  
}
```

```
.container { width: 600px; }  
.box { width: 50%; } /* 300px */
```

DISEÑO RESPONSIVO

Diseño flexible con rejillas (grid y flexbox)

- Usar **unidades relativas** (% , em, rem, vw, vh) en lugar de píxeles fijos.
- Estructurar los elementos con **CSS Grid o Flexbox** para que se adapten.

Flexbox (Flexible Box): Diseñado para **alinear y distribuir elementos dentro de un contenedor en una sola dimensión**: fila (row) o columna (column). Ideal para **componentes pequeños o alineación de elementos dentro de un contenedor**.

```
.container {  
  display: flex;           /* Activa Flexbox */  
  flex-direction: row;     /* dirección horizontal (fila) por defecto */  
  justify-content: center; /* alineación horizontal de los elementos */  
  align-items: center;     /* alineación vertical */  
  gap: 10px;              /* espacio entre elementos */  
}
```

```
.item {  
  flex-grow: 1; /* crece para llenar espacio disponible */  
  flex-shrink: 1; /* puede reducirse si hace falta */  
  flex-basis: 100px; /* tamaño base del elemento */  
}
```

DISEÑO RESPONSIVO

Diseño flexible con rejillas (grid y flexbox)

- Usar **unidades relativas** (% , em , rem , vw , vh) en lugar de píxeles fijos.
- Estructurar los elementos con **CSS Grid o Flexbox** para que se adapten.

CSS Grid: Diseñado para **layouts bidimensionales** (filas + columnas).
Ideal para **estructuras de página completas** o visualizaciones complejas.

```
.grid-container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr); /* 3 columnas iguales */  
  grid-template-rows: 200px 200px;      /* 2 filas de 200px */  
  gap: 10px;                             /* espacio entre filas y columnas */  
}
```

```
.item1 {  
  grid-column: 1 / 3; /* ocupa de la columna 1 a la 3 */  
  grid-row: 1 / 2;    /* ocupa la fila 1 */  
}
```

DISEÑO RESPONSIVO

Media queries

- Usar `@media` en CSS para definir estilos específicos según anchos de pantalla

Las **media queries** son reglas CSS que permiten **aplicar estilos diferentes según las características del dispositivo**, principalmente el ancho de pantalla, pero también otros factores como la resolución, orientación o tipo de pantalla.

Su objetivo: que tu diseño se **adapte dinámicamente** a distintos dispositivos sin necesidad de crear versiones separadas del sitio.

```
/* Estilo por defecto (desktop) */
body {
  font-size: 18px;
}

/* Para pantallas de 768px o menos (tablet y móvil) */
@media (max-width: 768px) {
  body {
    font-size: 16px;
  }
}
```

DISEÑO RESPONSIVO

Media queries: Ejemplos

Orientación del dispositivo:
portrait (vertical) y
landscape (horizontal)

```
@media (orientation: portrait) {  
  body {  
    background-color: lightblue;  
  }  
}
```

Resolución y densidad de
pantalla: Útil para pantallas
Retina y de alta densidad

```
@media (-webkit-min-device-pixel-ratio: 2), (min-resolution: 192dpi) {  
  img {  
    content: url(high-res-image.png);  
  }  
}
```

DISEÑO RESPONSIVO

Media queries: Estrategias comunes de diseño responsivo con media queries

Mobile first

Definir estilos base para móviles, luego usar **min-width** para tablet y desktop.

Desktop first

Definir estilos base para desktop y luego usar **max-width** para tablet/móvil.

```
body {  
  font-size: 14px;  
}  
  
/* Tablet */  
@media (min-width: 768px) {  
  body {  
    font-size: 16px;  
  }  
}  
  
/* Desktop */  
@media (min-width: 1024px) {  
  body {  
    font-size: 18px;  
  }  
}
```

DISEÑO RESPONSIVO

Tipografía adaptable

- Usar tamaños de letra relativos (**em**, **rem**, **clamp()**) que cambien con la pantalla.

em

Relativa al tamaño de fuente del elemento padre

```
.parent { font-size: 16px; }  
.child { font-size: 1.5em; } /* 16px * 1.5 = 24px */
```

rem

Relativa al tamaño de fuente raíz (<html>), no al elemento padre.

```
html { font-size: 16px; }  
p { font-size: 2rem; } /* 16px * 2 = 32px */
```

clamp()

Permite definir un valor que se ajusta dentro de un rango mínimo y máximo.

Sintaxis: `clamp(valor-mínimo, valor-preferido, valor-máximo)`

```
h1 {  
  font-size: clamp(1.5rem, 2.5vw, 3rem);  
}
```

FRAMEWORKS DE CSS RESPONSIVO



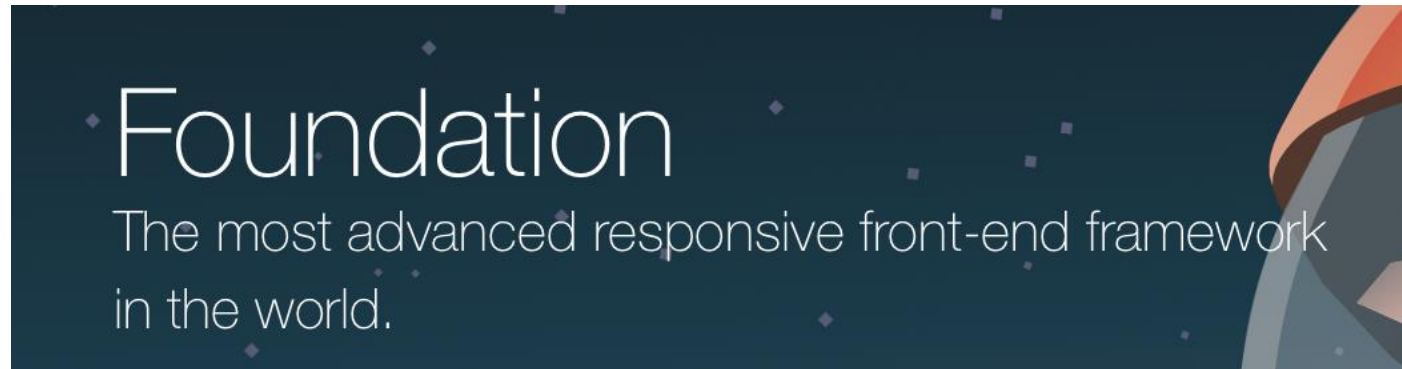
**Build fast, responsive sites
with Bootstrap**

```
<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">

<!-- Bootstrap JS (opcional, para componentes interactivos) -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

```
<div class="container">
  <div class="row">
    <div class="col-md-6">Columna 1</div>
    <div class="col-md-6">Columna 2</div>
  </div>
</div>
```

FRAMEWORKS DE CSS RESPONSIVO



```
<!-- Bootstrap CSS -->  
<link href="https://cdn.jsdelivr.net/npm/foundation-sites@6.8.1/dist/css/foundation.min.css" rel="stylesheet">  
  
<!-- Bootstrap JS (opcional, para componentes interactivos) -->  
<script src="https://cdn.jsdelivr.net/npm/foundation-sites@6.8.1/dist/js/foundation.min.js"></script>
```

```
<div class="grid-x grid-padding-x">  
  <div class="cell small-6">  
    <div class="cell small-6">Columna 1</div>  
  </div>
```

ETIQUETAS ARIA

ARIA significa Accessible Rich Internet Applications y son un conjunto de atributos HTML que ayudan a que las páginas web y aplicaciones sean más accesibles, especialmente para personas que usan lectores de pantalla u otras tecnologías de asistencia.

- Proveer información **semántica adicional** sobre elementos de la interfaz que **HTML estándar no comunica por sí mismo**.
- Mejorar la **navegación, comprensión y control** de elementos interactivos para todos los usuarios.



ETIQUETAS ARIA

Atributos ARIA comunes

Atributo ARIA	Qué hace
role	Define el tipo de elemento (por ejemplo: button, dialog, navigation).
aria-label	Proporciona un nombre accesible al elemento cuando no hay texto visible.
aria-labelledby	Indica qué elemento describe al actual, útil cuando hay títulos o etiquetas externas.
aria-hidden	Oculta contenido a los lectores de pantalla (true → no se lee).
aria-expanded	Indica si un elemento expandible (como un menú) está abierto (true o false).
aria-live	Permite que contenido dinámico sea anunciado automáticamente por lectores de pantalla.

